
DMC CHALLENGE · SOFTWARE DEVELOPMENT

Presentation Report

GrupoJap Rentals — Vehicle rental management web application

Technical report submitted with the DMC Challenge project, as part of the GrupoJap recruitment process.

Company	GrupoJap
Challenge	DMC — Software Development
Project	GrupoJap Rentals (administration panel)
Candidate	Nuno
Technologies	ASP.NET Core MVC (.NET 10), C#, Entity Framework Core 10, SQL Server, xUnit
Repository	github.com/nunoomigueIII/GrupoJap_Interview_Project

Contents

1.	Executive summary	3
2.	Challenge context and objectives	3
3.	Technology stack and rationale	3
4.	Solution architecture and code organisation	4
5.	Data model and database	6
6.	Features by module	7
7.	Validations and business rules	9
8.	Automatic vehicle status calculation	10
9.	Frontend and user experience	11
10.	Automated tests	12
11.	Version control (Git)	13
12.	Meeting the evaluation criteria	13
13.	Known limitations and future improvements	14
14.	Installation and running instructions	15
15.	Conclusion	15

1. Executive summary

GrupoJap Rentals is an administration panel built with ASP.NET Core MVC (.NET 10) that meets all four functional requirements of the challenge: vehicle registration, customer registration, rental contract registration, and lists that show each vehicle's Available/Rented status, calculated automatically. Persistence uses SQL Server with Entity Framework Core 10 in a Code-First approach, with 6 versioned migrations.

Every validation required by the brief is implemented on the server, enforced in the database (unique indexes) and mirrored on the client. Beyond the minimum, the solution adds:

- **Blocking of overlapping contracts** for the same vehicle, with inclusive dates — the rule that prevents a car from being rented twice.
- **Authentication and authorisation** with ASP.NET Core Identity, restricted to the Admin role, with account lockout after 5 failed attempts.
- **Home dashboard** with fleet indicators and recent contracts.
- **Interface in three languages** (PT, EN, ES), with 305 texts managed from within the application.
- **Dark mode, responsive design** and confirmation messages for every operation.
- **87 xUnit test methods** (≈133 cases including `InlineData`), covering business rules, validations and controllers.

The solution is organised into three projects — web application, core library and tests — with the business rules isolated in pure, testable classes.

2. Challenge context and objectives

The DMC Challenge asks for a web application for vehicle rentals, a real scenario from the automotive sector in which GrupoJap operates. The delivery deadline is 5 days, followed by a presentation to the panel explaining the technical choices and the architecture.

The brief defines four main features:

1. Vehicle registration (brand, model, registration plate, year of manufacture, fuel type).
2. Customer registration (full name, email, phone, driving licence).
3. Rental contract registration, linking a customer to a vehicle with start and end dates.
4. Lists of vehicles, customers and contracts, showing each vehicle's status (available or rented).

The technical requirements are ASP.NET MVC or ASP.NET Core MVC, C#, HTML/CSS/JavaScript, SQL Server and a data access mechanism (ADO.NET, Entity Framework or Dapper). Automated tests with xUnit or NUnit are valued as a differentiator.

My personal goal in this project was to deliver more than a CRUD: a coherent, secure and tested application, close to what would be put into production in a business setting.

3. Technology stack and rationale

The stack follows the brief's technical requirements to the letter, choosing at each point the most current

option and the one best suited to evaluation criterion 4 (Entity Framework Code-First).

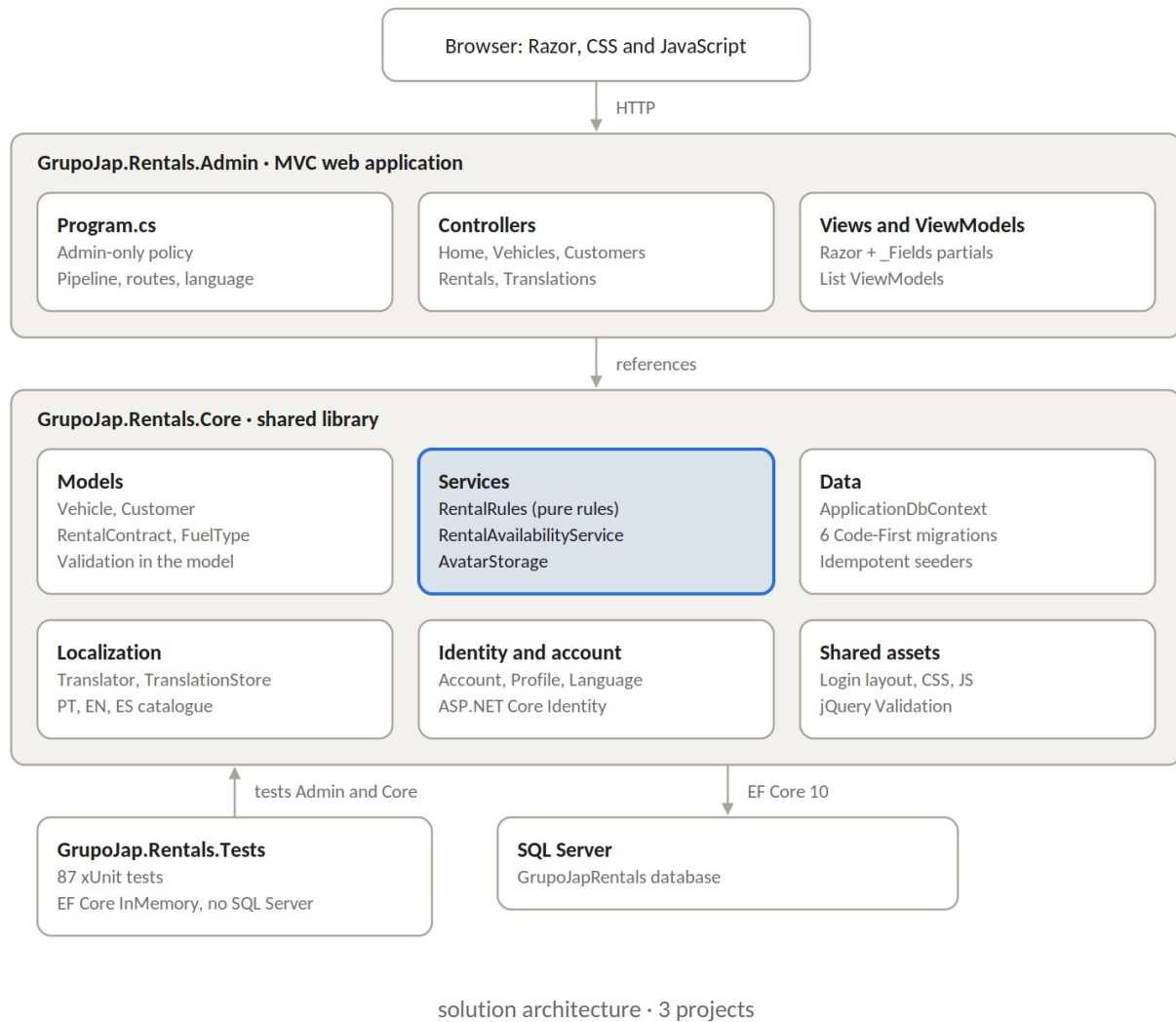
Layer	Technology	Why
Platform	ASP.NET Core MVC, .NET 10	Current, cross-platform version; the brief accepts any version.
Language	C# with Nullable enabled	Null warnings catch errors at compile time.
Data access	Entity Framework Core 10	ORM explicitly named in the evaluation criteria; typed LINQ queries.
Approach	Code-First with migrations	The C# model is the source of truth and the schema evolves in a versioned way.
Database	SQL Server (Express in development)	Required by the brief.
Authentication	ASP.NET Core Identity	Official solution, with password hashing, lockout and roles.
Frontend	Razor, HTML, custom CSS, plain JavaScript	Lightweight interface with no build dependencies.
Client-side validation	jQuery Validation + Unobtrusive	Reuses the same DataAnnotations as the server.
Tests	xUnit + EF Core InMemory	Framework accepted by the brief; fast tests without SQL Server.
Tooling	dotnet-ef 10.0.12 as a local tool	Version pinned in .config/dotnet-tools.json, the same for the whole team.

Unnecessary external dependencies were deliberately avoided. The whole interface (layout, tables, forms, dark mode) uses custom CSS and JavaScript, which keeps the project simple to run and to review.

4. Solution architecture and code organisation

The solution (GrupoJap.slnx) is split into three projects: the web application, a core library with the domain and infrastructure, and the tests. This separation keeps the business rules out of the controllers and allows them to be tested without a web server or SQL Server.

Business rules live in Core, shared and testable



Admin contains only what is specific to the management interface; everything reusable (models, data access, services, translations, login) lives in Core, a Razor Class Library that also shares views and static assets.

The MVC pattern in practice

- **Model**: domain entities with `DataAnnotations` and `IAvalidatableObject`; dedicated ViewModels for the lists, so entities are not exposed directly to views.
- **View**: Razor with Tag Helpers (`asp-for`, `asp-validation-for`), a shared layout and reusable partials (`_Fields`, `_Notices`, `_Eyebrow`).
- **Controller**: thin, sealed controllers with dependencies injected through primary constructors, attribute routing and asynchronous actions.

Architecture decisions

- **Pure rules separated from data access**: `RentalRules` is a static class with no dependencies (overlap, active contract, status). `RentalAvailabilityService` applies the same rule in an EF Core query.
- **Dependency injection**: all configuration sits in an `AddRentalsCore` extension method, which registers the `DbContext`, Identity, services and translations. `Program.cs` stays short and readable.
- **Secure by default**: a fallback policy requires the Admin role on every route; only login and access-denied

are anonymous. A new route is protected even if someone forgets the `[Authorize]` attribute.

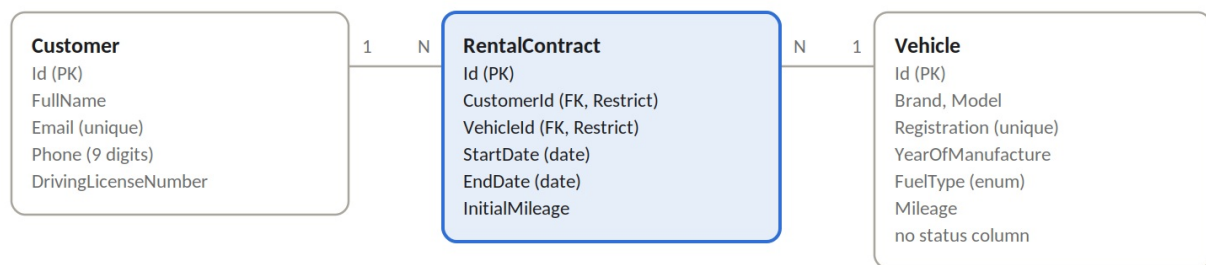
- **No generic repository layer:** EF Core's `DbContext` already implements the Unit of Work and Repository patterns. Another layer would add code with no benefit in a project of this size.

5. Data model and database

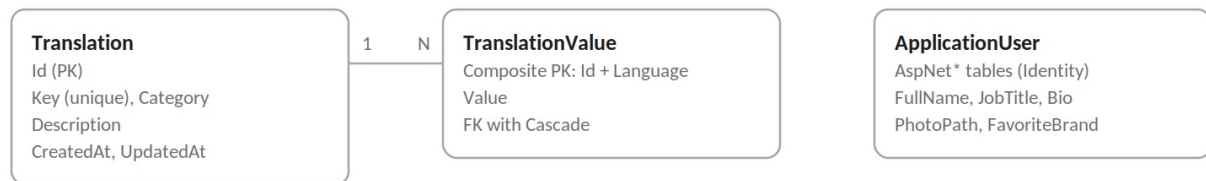
The model was defined in C# and generated in SQL Server with Entity Framework Core in Code-First: the classes are the source of truth and every schema change is a versioned migration.

Each contract links one customer to one vehicle; status is not stored

Challenge domain



Support: translations and identity



data model • 6 entities

A customer can have many contracts and a vehicle can have many contracts over time; each contract belongs to exactly one customer and one vehicle.

Modelling decisions

- **DateOnly for contract dates:** mapped to SQL Server's `date` type. A rental is counted in days, so storing times and time zones would only cause errors in comparisons.
- **Contract fields nullable in C# (`int?`, `DateOnly?`) but `[Required]`:** this distinguishes "field not filled in" from "zero value" and shows the user the right message, instead of a silent `0` or `01/01/0001`.
- **Fuel as an enum** (Petrol, Diesel, Hybrid, Electric, LPG, Other), stored as an integer. The value `Unspecified = 0` is never accepted, which forces an explicit choice.
- **Unique indexes** on `Vehicles.Registration`, `Customers.Email` and `Translations.Key`, declared with the `[Index]` attribute or the Fluent API.
- **DeleteBehavior.Restrict** on both contract foreign keys: deleting a customer or vehicle with a contract history is blocked by the database itself, preserving the history.
- **Length limits** (`StringLength`) on every text field, generating `nvarchar(n)` columns instead of `nvarchar(max)`.
- **Vehicle mileage:** an additional field (`Mileage`, 0 to 2,000,000 km), stored on the vehicle and used in the

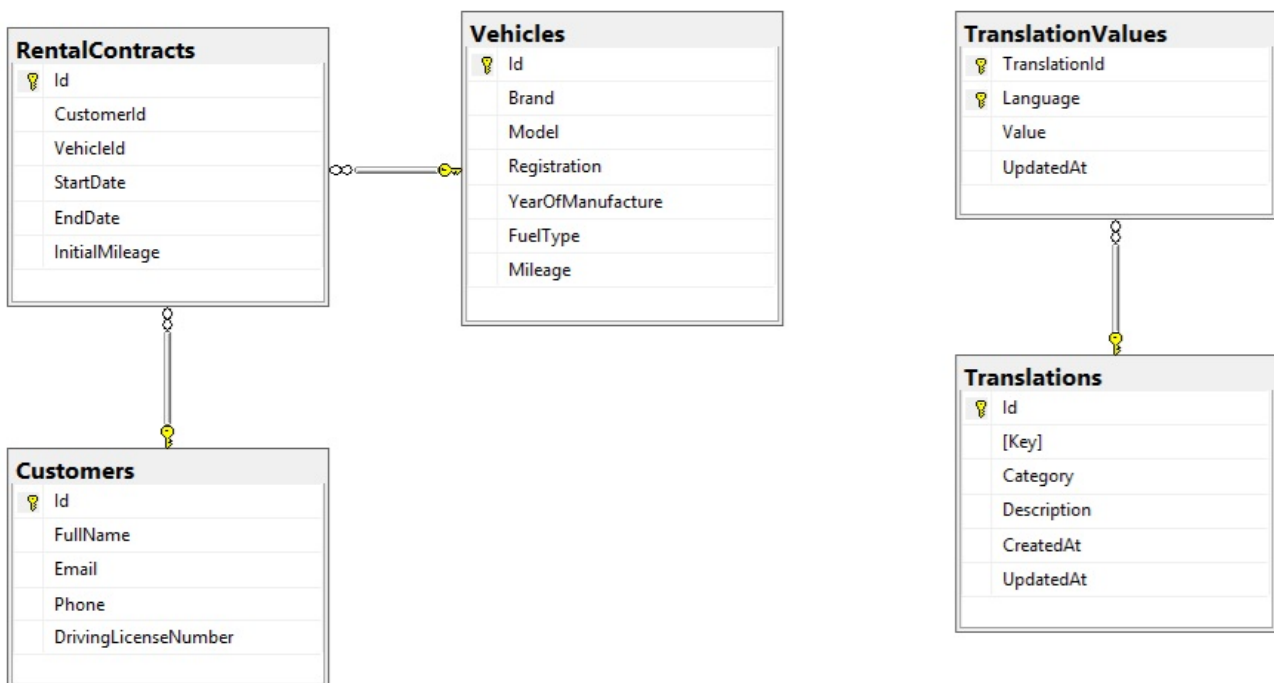
demo data as the contracts' initial mileage.

Migrations

The 6 migrations (`InitialCreate`, `LimitPhoneLength`, `AddIdentity`, `AddUserProfileFields`, `AddVehicleMileage`, `AddTranslations`) show the incremental evolution of the schema. The database can be recreated in any environment with a single `dotnet ef database update` command.

Actual schema in SQL Server

The following diagram was exported from SQL Server Management Studio from the `GrupoJapRentals` database, after the migrations were applied. It confirms that the schema generated by EF Core matches the model: `RentalContracts` links to `Customers` and `Vehicles` through foreign keys, `TranslationValues` depends on `Translations`, and the `AspNet*` tables belong to ASP.NET Core Identity. The `EFMigrationsHistory` table records the applied migrations.



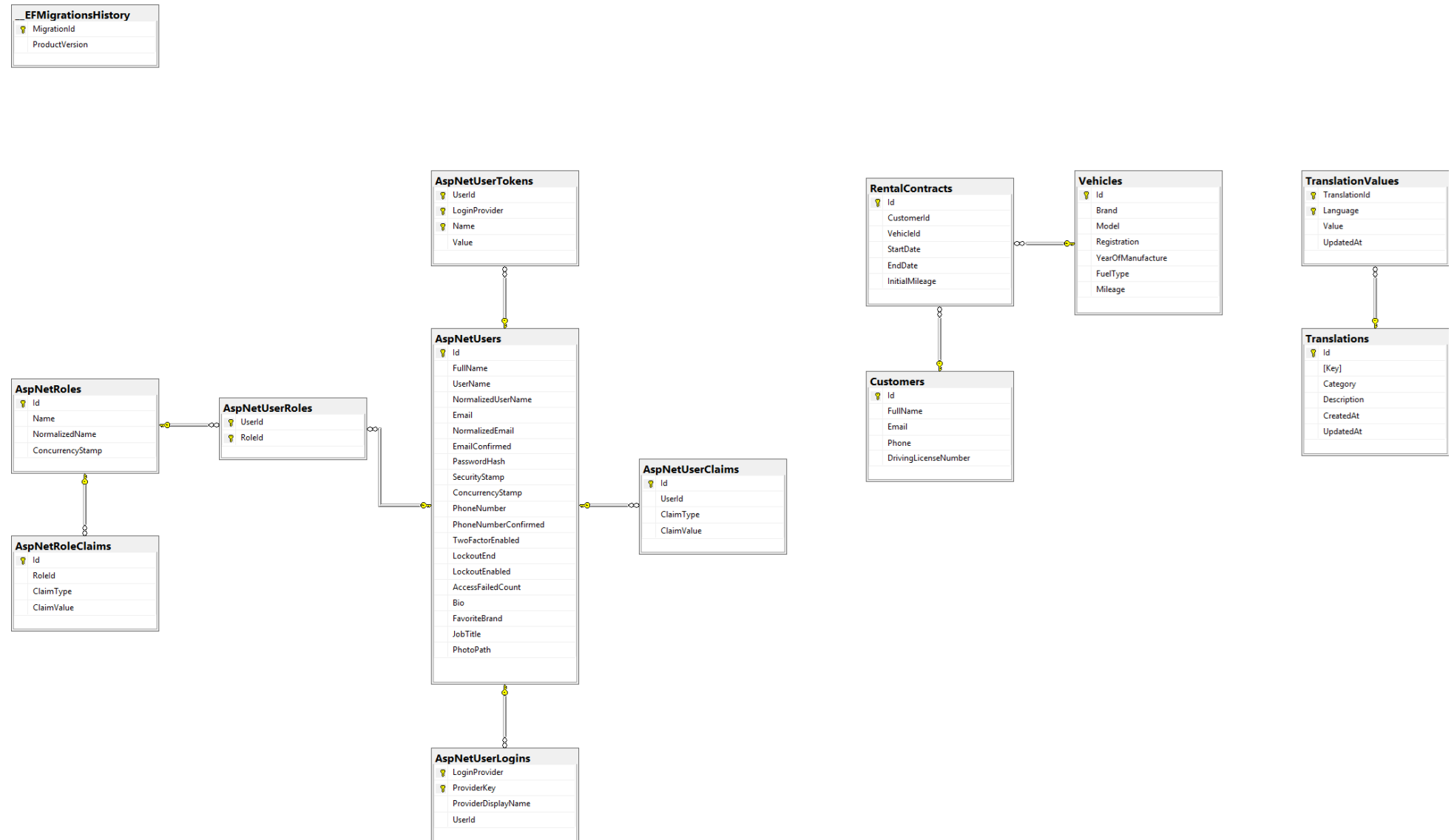
Domain and translation tables (excerpt from the diagram exported from SSMS). The full diagram, including the Identity tables, is on the next page.

6. Features by module

Each entity in the brief has a full CRUD — list, create, edit and delete with a confirmation page — with readable REST-style routes defined by attributes.

Module	Main routes	What it does
Dashboard	/	Total vehicles, available vehicles, active contracts, customers and the 5 most recent contracts.
Vehicles	/vehicles, /vehicles/create, /vehicles/{id}/edit, /vehicles/{id}/delete	Brand, model, registration, year, fuel and mileage; Available/Rented status in the list.
Customers	/customers, /customers/create, ...	Name, email, phone and driving licence; number of

Full E-R diagram of the GrupoJapRentals database (SQL Server Management Studio)



Includes the domain and translation tables, the ASP.NET Core Identity tables (AspNet*) and the migration history (__EFMigrationsHistory).

		contracts and whether a rental is in progress.
Contracts	/rentals, /rentals/create, ...	Links customer and vehicle with dates and initial mileage; Upcoming, Active or Completed status; instant search in the list.
Translations	/translations	Search, filter by category or missing texts, pagination and editing of texts in PT/EN/ES.
Account and profile	/account/login, /profile, /profile/password	Login, logout, profile with photo, password change.
Language	/language/{code}	Language switch stored in a cookie for 1 year.

Some details that improve day-to-day use:

- **Shortcuts between modules:** the contract form accepts `?vehicleId=` and `?customerId=` so the vehicle or customer arrives pre-selected.
- **Sensible defaults:** a new contract starts today and ends tomorrow; a new vehicle defaults to the current year.
- **Data normalisation:** registration plates and driving licences in upper case, emails in lower case, spaces removed from phone numbers. So `aa-00-bb` and `AA-00-BB` are treated as the same plate.
- **Referential integrity:** a vehicle or customer with associated contracts cannot be deleted. The confirmation page shows how many contracts exist and the database enforces the rule with `DeleteBehavior.Restrict`.
- **Feedback messages:** every operation ends with a success or error message (Post-Redirect-Get pattern with `TempData`).
- **Demo data:** in development, the application creates 14 vehicles, 3 customers and 5 contracts (active, upcoming and completed) if the fleet is empty, for an immediate demonstration.

7. Validations and business rules

All validations in section 4 of the brief are met, and each one is enforced on the server — JavaScript validation is only a usability complement, never the only protection.

Validation is organised in three layers:

1. **Model:** `DataAnnotations` (`[Required]`, `[Range]`, `[RegularExpression]`, `[EmailAddress]`) and `IValidatableObject` for rules that depend on the current date or on several fields.
2. **Controller:** rules that need to query the database — uniqueness, existence of the customer and vehicle, contract overlap.
3. **Database:** unique indexes on `Registration` and `Email` and foreign keys with `Restrict`. If two simultaneous requests pass validation, SQL Server rejects the second; the application catches errors 2601/2627 and shows the same friendly message.

Requirement → implementation mapping

Requirement in the brief	Implementation	Where
Vehicle: brand, model, registration required	<code>[Required]</code> + length limits	<code>Vehicle</code>
Vehicle: year and fuel	<code>[Range(1, ...)]</code> on the year; the enum's	<code>Vehicle.Validate</code>

required	Unspecified value is rejected	
Year of manufacture not later than the current year	Comparison with <code>DateTime.Today.Year</code>	<code>Vehicle.Validate</code>
Unique registration	Prior check + unique index + SQL error handling	<code>VehiclesController</code> , <code>[Index(IsUnique)]</code>
Customer: all fields required	<code>[Required]</code> on name, email, phone and licence	<code>Customer</code>
Unique email	Normalised to lower case, prior check + unique index	<code>CustomersController</code>
Valid email format (extra)	<code>[EmailAddress]</code>	<code>Customer</code>
Phone with digits only and a valid format	Pattern <code>^9[0-9]{8}\$</code> (Portuguese mobile: 9 digits starting with 9)	<code>PhoneNumbers.PortuguesePattern</code>
Contract: customer and vehicle required	<code>[Required]</code> + check that they exist in the database	<code>RentalContract</code> , <code>RentalsController</code>
Dates and initial mileage required	<code>[Required]</code> ; <code>mileage >= 0</code>	<code>RentalContract</code>
Start date not earlier than today	Comparison with today (<code>DateOnly</code>)	<code>RentalContract.Validate</code>
End date after the start date	<code>end <= start</code> is rejected	<code>RentalContract.Validate</code>
Automatic vehicle status	Calculated from the contracts (see section 8)	<code>RentalRules</code> , lists

Additional rules beyond the brief

- **No overlapping contracts:** a vehicle cannot have two contracts whose periods intersect (details in section 8).
- **Editing contracts already in progress:** an active contract can be corrected (for example, extending the end date) as long as the original start date is kept. Without this rule, the "start not earlier than today" validation would block any edit of active contracts.
- **Over-posting protection:** every POST action uses `[Bind]` with an explicit list of accepted fields.
- **CSRF protection:** every POST form uses `[ValidateAntiForgeryToken]`.
- **Translated messages:** error messages are translation keys, so they appear in the chosen language.

8. Automatic vehicle status calculation

The Available/Rented status is not stored in the database: it is calculated on every query from the contracts. This is the project's central technical decision, because it removes by design the risk of the status becoming stale.

Why not store the status

An `IsRented` column on the vehicles table would require a process (scheduled job, trigger or manual update) to change it when a contract starts or ends. If that process failed or ran late, the vehicle would show the wrong status. By deriving the status from the contract dates, the information is always correct at the moment it is displayed, with no background process.

The rules, in one place

The rules live in the static class `RentalRules`, with no data access, which makes them easy to read and test. All dates are inclusive and use `DateOnly`, which represents a day with no time or time zone.

```
// A contract is active on a given day when start <= day <= end
public static bool IsActiveOn(DateOnly start, DateOnly end, DateOnly day)
    => start <= day && end >= day;

// Two inclusive periods overlap when startA <= endB and endA >= startB
public static bool PeriodsOverlap(DateOnly startA, DateOnly endA, DateOnly startB,
    DateOnly endB)
    => startA <= endB && endA >= startB;
```

- **Vehicle:** Rented if there is at least one contract with `start <= today <= end`; otherwise, Available.
- **Contract:** Upcoming if it has not started yet, Active if today falls within the period, Completed if it has ended (`RentalStatus` enum).
- **Customer:** the list shows whether the customer has a rental in progress.

Efficiency

In the lists, the status is calculated inside the SQL query itself: the LINQ expression

`vehicle.RentalContracts.Any(r => r.StartDate <= today && r.EndDate >= today)` is translated by EF Core into an `EXISTS` subquery. The full list is therefore retrieved in a single database round trip, without the N+1 problem (one extra query per vehicle). Read queries use `AsNoTracking()` and project directly into ViewModels, loading only the columns needed.

Preventing overlaps

The same overlap formula is used by `RentalAvailabilityService` when creating or editing a contract. The check is an `AnyAsync` in SQL Server (a single database round trip, supported by the index on the `VehicleId` foreign key). When editing, the contract itself is excluded from the comparison so it does not conflict with itself.

Example with inclusive dates: if a vehicle is rented from the 10th to the 15th, a new contract from the 15th to the 20th is rejected (the 15th is shared), but one from the 16th to the 20th is accepted.

9. Frontend and user experience

The interface was designed as a fleet management panel, with a visual identity inspired by GrupoJap and no CSS frameworks: about 460 lines of custom CSS and 86 lines of custom JavaScript.

- **Layout:** side menu with active navigation, top bar with language selector, dark mode, current date and the user's avatar.
- **Responsive:** on small screens the side menu becomes a drawer that closes with its button, with Escape or by clicking outside; tables have their own horizontal scroll.
- **Dark mode:** follows the system theme by default and remembers the user's choice in the browser. The theme is applied before the page is drawn, to avoid a colour "flash".
- **Visual states:** coloured badges for Available, Rented, Upcoming, Active and Completed; registration plates styled like number plates; empty states with a direct link to create the first record.

- **Forms:** fields shared between Create and Edit through partials (`_Fields.cshtml`), immediate client-side validation and messages next to each field.
- **Instant search** in the contracts list, by customer, vehicle or plate, without reloading the page.
- **Accessibility:** `aria-label`, `aria-expanded` and `aria-pressed` on controls, decorative icons with `aria-hidden` and keyboard navigation.

Internationalisation

The panel is available in Portuguese (base language), English and Spanish. Instead of `.resx` files, the 305 texts live in the database (`Translations` and `TranslationValues` tables) and can be edited in the panel itself, with no new deployment.

- A catalogue in code (`TranslationCatalog`) defines the initial values; at startup only missing keys are inserted, without overwriting edits made by users.
- An in-memory cache (`TranslationStore`) avoids querying the database for every text. It is discarded when a change is saved and expires on its own after 10 seconds. Reloading uses a double-checked lock, safe under concurrent requests.
- Fallback order for each text: database in the current language → catalogue in the current language → database in Portuguese → catalogue in Portuguese → the key itself. The interface never shows empty texts.
- `Translator` replaces the framework's `IStringLocalizerFactory`, so `DataAnnotations` messages are translated too.

10. Automated tests

The project includes 87 xUnit test methods — 70 [Fact] and 17 [Theory] with 63 data sets — for a total of about 133 executed cases. The tests do not need SQL Server: each one uses a fresh EF Core in-memory database, so they are independent of each other and run in seconds.

Test file	Tests	What it guarantees
<code>RentalsControllerTests</code>	16	Period conflicts at the boundaries, adjacent periods accepted, unknown customer/vehicle, start in the past, invalid end, negative mileage, editing without conflicting with itself, contracts already started
<code>ModelValidationTests</code>	13	Required fields, year not in the future, fuel, phone, email, dates and mileage
<code>TranslatorTests</code>	10	Translation fallback order and cache
<code>VehiclesControllerTests</code>	9	Duplicate registration on create and edit, normalisation, blocked deletion, status in the list
<code>CustomersControllerTests</code>	9	Case-insensitive duplicate email, invalid email and phone, blocked deletion
<code>TranslationCatalogTests</code>	6	Complete catalogue in all 3 languages, consistent <code>{0}</code> placeholders, every key used in code exists
<code>TranslationSeederTests</code>	5	The seeder does not overwrite edits and removes obsolete keys
<code>AvatarStorageTests</code>	5	Detection of the real file type and size limit
<code>LanguageControllerTests</code>	4	Language cookie and open-redirect protection

RentalRulesTests	3	Overlap and active contract with inclusive dates, Upcoming/Active/Completed status
HomeControllerTests	3	Dashboard indicators and recent contracts
ValidationLocalizationTests	2	Translated validation messages
RentalAvailabilityServiceTests	2	Overlap-checking service

Controller tests use a dedicated context (`ControllerTestContext`) that builds the controller as in a real request: in-memory database, real translations and TempData. It also reproduces the model validation MVC runs before calling the action, so the full flow is tested, not just isolated methods.

The `[Theory]` tests are used mainly at date boundaries, where bugs tend to hide: periods sharing a single day, adjacent periods and end dates equal to the start date.

Command to run them: `dotnet test`.

11. Version control (Git)

The project is versioned in Git on the `nuno-dev` branch, with small, descriptive commits throughout development, as defined in the work plan in the README.

Repository: github.com/nunoomiguell/GrupoJap_Interview_Project

Good practices applied:

- **.gitignore** excludes `bin/`, `obj/`, `.vs/`, `*.user` files and the `uploads/` folder with user photos.
- **Versioned migrations:** the schema history travels with the code, so the database can be recreated in any environment.
- **Pinned tooling:** `dotnet-ef` as a local tool, so everyone uses the same version.
- **Credentials kept out of shared code:** the README recommends User Secrets or environment variables outside the development environment.

The 5-day plan was followed in phases, which is reflected in the order of the migrations:

Date	Migration	Phase
3 Oct 2026	AddTranslations	User experience (languages)
2 Oct 2026	AddVehicleMileage	Contracts (mileage)
2 Oct 2026	AddUserProfileFields	User profile
2 Oct 2026	AddIdentity	Authentication
1 Oct 2026	LimitPhoneLength	Customers (phone)
1 Oct 2026	InitialCreate	Domain and persistence

12. Meeting the evaluation criteria

The table links each criterion in section 6 of the brief to concrete evidence in the project.

Criterion	Evidence in the project
1. C# and .NET	.NET 10, modern C#: primary constructors, <code>sealed</code> , <code>record</code> , <code>DateOnly</code> , pattern matching, <code>Nullable</code> enabled, <code>async/await</code> for all data access.

2. Data structures and algorithms	Interval overlap in $O(1)$ per pair; status calculated with <code>EXISTS</code> in a single query; translation cache in dictionaries ($O(1)$ lookup) with a double-checked lock.
3. Web development with ASP.NET	Readable attribute routes (<code>/vehicles/{id}/edit</code>), type constraints (<code>{id:int}</code>), PRG with <code>TempData</code> , <code>ViewModels</code> , reusable partials, Tag Helpers.
4. SQL Server and Entity Framework	Code-First with 6 migrations, unique indexes, foreign keys with <code>Restrict</code> , composite key, projections and <code>AsNoTracking()</code> .
5. Git	History of small commits, <code>.gitignore</code> , versioned migrations and tooling (section 11).
6. OOP	Entities encapsulate their own validation (<code>IValidatableObject</code>); pure rules in <code>RentalRules</code> ; injected <code>RentalAvailabilityService</code> ; <code>sealed</code> classes by default; configuration extension methods.
7. Problem analysis	Phased plan in the README; documented decisions (phone format, inclusive dates, derived status).
8. Algorithmic reasoning	A single overlap formula, tested at the boundaries; editing of active contracts handled as a special case.
9. Critical thinking and creativity	Overlap prevention, authentication, editable multi-language support, dark mode, dashboard, demo data.
10. Automated tests (differentiator)	87 xUnit methods, ≈ 133 cases, including controller tests with an in-memory database.

13. Known limitations and future improvements

Within the 5-day deadline, the brief's requirements and the quality of the business rules were prioritised. The points below are deliberate limitations, each with the solution that would be adopted next.

Current limitation	Impact	Proposed improvement
The overlap check runs in the application, not in the database	Two exactly simultaneous requests for the same vehicle could, in theory, both pass	Transaction with <code>Serializable</code> isolation or a per-vehicle lock (<code>sp_getapplock</code>) when creating/editing contracts
Tests use EF Core InMemory	They do not validate unique indexes or the exact SQL translation	Integration tests against a real SQL Server (Testcontainers or LocalDB)
No contract closing (vehicle return)	Final mileage is not recorded and the vehicle's mileage is not updated	"Return vehicle" action with final km, validation $\text{final km} \geq \text{initial km}$, and vehicle update
Initial mileage is not compared with the vehicle's	A value lower than the current one can be entered	Default to the vehicle's km and validate that it is not lower
Phone accepts only Portuguese mobiles (9 digits starting with 9)	Excludes landlines and international numbers	<code>Libphonenumber</code> library with country code
Driving licence has no format or expiry validation	Accepts any text up to 30 characters	Validate format and licence expiry at the contract date
No pagination or filters in	Slow with thousands of records	Server-side pagination and filters by status, brand

the vehicle and customer lists		and fuel (already present in translations)
"Today" uses the server clock	Depends on the server's time zone	Inject <code>TimeProvider</code> (.NET 8+), which also makes testing easier

Other natural evolutions for the business:

- **Pricing and invoicing:** daily rate per vehicle and calculation of the contract value.
- **Availability calendar:** monthly view per vehicle with the scheduled contracts.
- **Audit trail:** record of who created or changed each contract (`CreatedBy`, `UpdatedAt`).
- **More explicit layered architecture:** if the domain grows, separate application services from the controllers (for example, `VehicleService`) to reduce logic in controllers.
- **CI/CD:** a GitHub Actions pipeline to build and run the tests on every push.

14. Installation and running instructions

Prerequisites: .NET 10 SDK and SQL Server (Express or another instance). The development connection string uses `localhost\SQLEXPRESS` with Windows authentication and can be changed in `src/GrupoJap.Rentals.Admin/appsettings.Development.json`.

1. Restore packages and tools: `dotnet restore` and `dotnet tool restore`.
2. Create the database: `dotnet ef database update --project src/GrupoJap.Rentals.Core --startup-project src/GrupoJap.Rentals.Admin`.
3. Start the application: `dotnet run --project src/GrupoJap.Rentals.Admin`.
4. Open `http://localhost:5169` and sign in with the development administrator (`admin@admin.com`).
5. Run the tests: `dotnet test`.

On first start, the roles, the administrator, the translation texts and the demo data are created automatically. All these operations are idempotent: they do not duplicate or change existing data.

Suggested demo script

1. Log in and show the dashboard with the fleet indicators.
2. Vehicle list: Available and Rented statuses already calculated from the demo contracts.
3. Create a vehicle with a duplicate plate or a future year, to show the validations.
4. Create a customer with a duplicate email or an invalid phone.
5. Create a contract overlapping another one for the same vehicle and see it rejected; then a valid contract.
6. Go back to the vehicle list and see the updated status.
7. Switch the language to English, turn on dark mode and edit a translation.
8. Run `dotnet test` and show the tests passing.

15. Conclusion

GrupoJap Rentals meets all the functional and technical requirements of the DMC Challenge, plus the automated tests differentiator. The decision to derive vehicle status from the contracts, instead of storing it, guarantees that the information shown is always correct.

Beyond what was asked, the application prevents overlapping rentals, protects access with authentication and roles, and offers an editable, responsive multi-language interface. The business rules are isolated and covered by about 133 test cases.

The limitations identified in section 13 are known and have a clear path forward. The project was built as a solid starting point for a real rental management system at GrupoJap.